BOOTCAMP LAB TEST PLUGIN

mcs-lab-auditor

Audit and build Microsoft Copilot Studio workshop labs, end-to-end.

A Claude Code plugin that drives the live product UI with Playwright and judges each step with an LLM. **Audit mode** files GitHub issues + matching fix PRs against `microsoft/mcs-labs` whenever the lab and the UI disagree. **Build mode** interactively authors a new lab step-by-step, gates it through the same audit engine, and opens a PR adding it. Both modes are event/workshop-agnostic.

Overview · Architecture · Installation · CUA comparison | v0.4.0 · 2026-06-04

CONTENTS

1 **Overview**

audit + build modes, top-level entry points

2 **Why a CUA cannot replace this**

interview, correction loop, verdicts, static checks, PR authoring

3 **Architecture**

boundaries, components, run lifecycle, cross-lab consistency

4 **Installation**

prerequisites, step-by-step, first run, Claude Code + Copilot CLI

5 **Configuration**

workshop.yml + judge-config.yml keys

6 **Troubleshooting**

common failure modes

7 **Known limitations**

what this plugin will not do

8 **Projected token spend**

per-step / per-lab / per-event estimates + cost knobs

9 **Read next**

where to find the operational rulebooks

SECTION 1

WHAT THE PLUGIN DOES

Overview

mcs-lab-auditor has two modes. **Audit mode** takes a workshop event (or a single lab) and produces, per lab, either (a) a clean-pass entry in a local audit log, or (b) one GitHub issue plus one fix-PR against `microsoft/mcs-labs` describing every step where the live UI did not match the written instruction. **Build mode** (v0.4.0+) interactively authors a brand-new lab, drives every step through Playwright, gates the finished lab through the same audit engine, and opens a PR adding it.

Both modes are **event/workshop-agnostic**. Any entry in `_data/lab-config.yml.event_configs` (Architecture Bootcamp, Agent Build-A-Thon variants, MCS-in-a-Day variants, the Azure AI workshop, anything added later) is a valid audit scope; single labs can be audited against the full `lab_metadata` catalog; and a built lab is created standalone with optional event attachment - nothing is hardcoded to bootcamp.

Top-level entry points

Command	Purpose
/audit-event [--event]	Audit every lab in a workshop event. With --event, pinned; without, the interview asks.
/audit-bootcamp	Shortcut for /audit-event --event bootcamp.
/audit-lab []	Audit a single lab. With , scope is pinned. Without, the interview picks from the full all-labs catalog.
/audit-report []	Print a local summary of recent audit runs. No browser activity.
/audit-account [show|redeem|clear]	Manage the DPAPI-cached workshop-issued test account.
/build-lab []	Interactively build a NEW lab end-to-end, gate it through the audit engine, and open a PR (v0.4.0+).

What runs at a glance (audit mode)

- **Run-start interview** (Q1 account / Q2 phase mix / Q3 scope / Q3a event picker / Q4 lab picker).
- **Lab parsing** into use cases, scenes, and numbered steps. Image refs become semantic hints.
- **Static fan-out + cross-lab consistency** (markdown checks, link checks, image-ref resolution, identifier-token drift between sibling labs).
- **Interactive step execution** via Playwright MCP - one per-Use-Case subagent at a time.

- **LLM judge** emits a structured verdict per step (pass / broken / unclear / non_deterministic / transient / cannot_verify).
- **Issue + fix-PR, or local log**. Findings render to one issue body and one PR with the literal diffs applied.

Build mode at a glance (/build-lab)

Build mode reuses the account flow, Playwright cookbook, judge, and finding schema above, and adds an interactive authoring loop (phases B0-B7). It confirms **every** step with the user.

- **Preflight + interview** - assert Opus, check gh, resolve the mcs-labs path, detect the registration mechanism; pick the account and a mode: **guided** (you dictate each step) or **scenario** (the AI proposes each step).
- **Navigate** to the Copilot Studio Home page; name the lab (slug + collision check); capture metadata + optional event attachment.
- **Capture loop** - snapshot, execute the step in Playwright, screenshot, write the instruction + tips, confirm, checkpoint to a ledger.
- **Assemble** the sibling-formatted labs/<slug>/README.md from the ledger.
- **Audit gate** - register + run the audit engine against the built lab with all GitHub writes suppressed; broken/unclear steps loop back for a fix until it passes.
- **New-lab PR** - add labs/<slug>/README.md + screenshots + the registration entry. Skipped under --no-pr.

SECTION 2

DESIGN RATIONALE

Why a generic Computer-Use Agent (CUA) cannot replace this plugin

A general-purpose CUA - an agent that observes a screen, plans clicks, and manipulates the OS through a single screenshot-and-action loop - is the closest off-the-shelf alternative. It is **not a substitute**. The plugin combines structured pre-processing, dynamic verification against a known specification, and downstream code authorship in ways a CUA pipeline does not.

WHY THIS MATTERS

The gaps below are not implementation oversights in current CUAs - they are categorically outside the CUA design contract. A CUA receives a screen and a task, then optimizes actions. It does not receive a typed lab specification, an issue tracker, or a git working tree, and it does not maintain a per-step verdict log.

2.1 No pre-process interview before destructive work

The plugin runs a Phase 1.5 run-start interview before touching anything: it confirms the account, the phase mix (static / interactive / both), the scope (event vs single lab), the chosen event, and the chosen lab. Every question is mandatory unless explicitly skipped by a CLI flag. This safety net catches the "wrong tenant" class of failure that has already cost real audit time.

A CUA invokes immediately on whatever task description it receives. It has no protocol for pausing and asking the user "which of these six events did you mean?" before opening the browser, and no structured way to record the answer in a manifest the next run can resume from.

2.2 No interactive problem-solving / correction loop

When the auditor encounters something wrong mid-run, the human can redirect the agent ("save it before navigating away," "don't include that NOTE," "update the actual PNG, not just alt text") and the agent updates its working approach in place. This is interactive problem-solving against the same browser session and the same in-progress fix branch.

A CUA's natural unit is an isolated task with a screen-grounded objective, not a long-running, redirectable collaboration where corrections compound on a working set of files.

2.3 No dynamic verification of screenshots against a typed lab spec

For each numbered step, the plugin captures a before-snapshot, dispatches the step, captures an after-snapshot and a screenshot, and feeds all three to an LLM judge alongside the raw markdown instruction. The judge returns a structured verdict (**pass / broken / unclear / non_deterministic / transient / cannot_verify**) with confidence and a `suggested_correction` that points at the literal text in the lab to replace.

A CUA watches a screen and acts. It does not have a typed specification (parsed lab markdown, expected visual references, alert blocks attached as hints) to verify against, and it does not produce structured per-step verdicts that an issue body can be rendered from.

2.4 No static-phase analysis

Roughly a third of real findings never appear on a screen: typos, broken external URLs, anchor-hash mismatches, image references that point at deleted files, front-matter versus body-table divergences, and cross-lab drift in column names or control labels. The plugin's static fan-out runs one subagent per lab against the raw markdown, then a fan-in pass groups scenes across labs by shape hash and emits drift findings - e.g. `Address 1: State/Province` in one lab versus `Address1: State or Providence` in a sibling lab.

> A CUA never reads the markdown source. It only sees what the browser renders. Static findings - typos, drift, broken refs - are invisible to it by construction.

2.5 No GitHub issue or PR authorship

The plugin closes the loop. For every lab with findings, it (a) files a GitHub issue (or fingerprint-dedup-comments on the existing one) describing every finding with a literal diff, evidence path, and severity tag, and (b) opens a fix-PR on branch `dewain/fix-<slug>-content-audit` that applies the `suggested_correction` diffs to the lab markdown and copies any refreshed screenshots into `labs/<slug>/images/`.

A CUA does not write source files or open pull requests. Even a CUA wired to a code editor would lack the per-finding fingerprint dedup and the same-author / mergeable / unprotected-branch guardrails the plugin enforces.

2.6 No structured resume / context-window management

An 11-lab bootcamp generates more accessibility snapshots, screenshots, console messages, and network records than a single agent's context window can hold. The plugin slices each lab into per-Use-Case subagents (5-20 steps each), writes `uc-<N>-state.yml` handoffs, and resumes interrupted runs at the first UC missing a state file.

A CUA is a single agent with a single rolling context. It has no equivalent of `--resume <run-id>` against per-UC checkpoints, and a long audit hits its context limits long before the run completes.

> A CUA is good at executing a single screen-bound task. The plugin is structured around the entire audit pipeline - pre-process interview, lab-aware parsing, dynamic verification with structured verdicts, static checks the screen never reveals, and downstream issue+PR authorship - and the value **compounds across those stages**. Replacing the pipeline with one CUA loop would drop everything except step 2.3, and even that loses the structured verdict it depends on.

SECTION 3

HOW IT WORKS

Architecture

The plugin has no compiled code and no test runner. Claude is the runtime; the plugin is a tree of markdown (commands, skills, references) and YAML configuration. It orchestrates three external systems: the user's filesystem (the cloned `mcs-labs` repo), the Playwright MCP (a real browser against Microsoft product portals), and the GitHub Issues + PRs APIs.

Boundaries

- **Source-of-truth boundary** - `_data/lab-config.yml` provides the event catalog (`event_configs.<key>`) and the all-labs catalog (`lab_metadata.*`). Slug lists are never hard-coded in the plugin.
- **Write boundary** - three narrow paths: (i) `gh issue create | comment | edit`; (ii) `gh pr create` on per-slug fix branches; (iii) `git push` of one screenshots-only commit onto an already-open fix-PR.
- **Secret boundary** - workshop credentials live only in `runtime/account/credential.enc` (DPAPI-encrypted, current-user scope) and in memory for one sign-in dispatch.

Components

Component	Role
commands/	Slash command entry points (audit-event, audit-bootcamp, audit-lab, audit-report, audit-account).
skills/mcs-lab-auditor/	Primary orchestrator: pre-flight, interview, parse, dispatch, judge, file.
skills/mcs-lab-issue-filer/	Render findings.json into an issue body and create or comment via gh.
skills/mcs-lab-fix-pr-filer/	Apply suggested_correction diffs to lab files, commit, push, open PR.
skills/mcs-lab-pr-appender/	Push a screenshots-only commit onto an already-open fix-PR branch.
references/	Operational rulebooks loaded on demand.
config/workshop.yml	Workshop portal URL + redemption selectors.
config/judge-config.yml	Confidence thresholds, retry caps, dedup config, cross-lab consistency knobs.
runtime/account/	DPAPI-encrypted credentials + non-secret account metadata.
runtime/runs//	Per-run parsed steps, findings, screenshots, transcripts, issue bodies.
runtime/audit-history.yml	Rolling local log of every audit run, pass or fail.

Run lifecycle

Phases reference `skills/mcs-lab-auditor/SKILL.md`:

Phase	What happens
1	Pre-flight: load configs, build events map + all-labs catalog, check gh auth.
1.4	Existing-state probe: gh issue list + gh pr list per slug. Output: existing-state.yml.
1.5	Run-start interview (Q1-Q4).
1.6	Lab Resources discovery + pre-flight scrape (conditional).
1.7	Plan execution order. Static fan-out per lab; step 1a: cross-lab consistency fan-in.
2	Interactive per-lab loop. Each lab is sliced into per-Use-Case subagents.
3	Wrap-up: close browser, print summary, save manifest.

Per-step data flow

Inside one step of one scene of one lab:

```
step (from steps.json)
    -> _browser_snapshot (before)
    -> dispatch by kind  (navigate / click / type / select / wait / inspect)
    -> _browser_snapshot (after) + _browser_take_screenshot
    -> _browser_console_messages + _browser_network_requests
    -> judge prompt (per-step)
         outcome:    pass | broken | unclear | non_deterministic |
                     transient | cannot_verify
         confidence: 0..1
         suggested_correction: { original_text, proposed_text,
                                 rationale, scope }
    -> critique pass (optional second opinion)
    -> append to findings.json (unless pass)
```

Cross-lab consistency check

After the static fan-out completes, a single fan-in pass groups scenes by shape hash across the lab set and emits drift findings for divergent identifier tokens. Sibling labs that verify the same UI surface get their wording aligned. Findings are severity `low` with `flags.cross_lab_drift: true` and render under a dedicated "Cross-lab consistency" section in issue bodies.

LIVE EXAMPLE

Discovered between `mcs-multi-agent` UC3 and `mcs-orchestration` UC1 during the 2026-05-26 audit cycle: same Account Data Lookup Agent verification flow, but one lab said `Address1: State or Providence` and the other (correct) said `Address 1: State/Province`. The cross-lab check now catches this class of drift at fan-in time.

SECTION 4

FROM CLONE TO FIRST AUDIT

Installation

Windows-only (DPAPI dependency). Targets PowerShell 7+ and Claude Code with the global Playwright MCP plugin enabled.

Prerequisites

- Windows 10 or 11 (DPAPI is required for credential storage).
- `gh` CLI authenticated against `microsoft/mcs-labs`.
- PowerShell 7+ (`$PSVersionTable.PSVersion.Major` >= 7).
- Claude Code with the Playwright MCP plugin enabled (`playwright@claude-plugins-official`).
- A local clone of `microsoft/mcs-labs` at `C:\Users\<you>\mcs-labs`.
- An unredeemed workshop code for the event you plan to audit.

Step 1 - Clone the plugin

The plugin is a directory of markdown and YAML - no compiled artifacts. Both **Claude Code** and **GitHub Copilot CLI** auto-discover plugins from their respective user-plugins directories; install via `git clone` into the right place. Pick whichever runtime(s) you want.

Option A — Claude Code (primary, fully tested)

```
git clone https://github.com/microsoft/BootcampLabTestPlugin `
    "$env:USERPROFILE\.claude\plugins\mcs-lab-auditor"
```

Option B — GitHub Copilot CLI

Same skill-discovery model, different plugins directory. Find the directory with `copilot --help` (look for the *plugins* or *extensions* section) or `copilot config get plugins.path` if your version supports it. Then:

```
# Example - confirm $copilotPluginsPath against `copilot --help` first
git clone https://github.com/microsoft/BootcampLabTestPlugin `
    (Join-Path $copilotPluginsPath "mcs-lab-auditor")
```

> Some command files hard-code `$env:USERPROFILE\.claude\plugins\mcs-lab-auditor` in their preflight `!` interpolations. Adjust to your Copilot CLI plugins directory or use a symlink (see Option C). Workshop-portal redemption needs Playwright MCP tools - if your Copilot CLI session lacks them, use `--static-only` for doc-audit sweeps and fall back to Claude Code for the interactive phase. DPAPI is Windows-only regardless of runtime.

Option C — both at once

Symlink (or hardlink) the Copilot CLI directory at the Claude Code clone so both runtimes share one source of truth:

```
# Elevated PowerShell required for directory symlinks
New-Item -ItemType SymbolicLink `
    -Path (Join-Path $copilotPluginsPath "mcs-lab-auditor") `
    -Target "$env:USERPROFILE\.claude\plugins\mcs-lab-auditor"
```

Step 2 - Restart your runtime

Plugins are discovered at session start. Close Claude Code (or end your Copilot CLI session) completely and reopen. Type `/` in the prompt and verify these commands appear: `/audit-event`, `/audit-bootcamp`, `/audit-lab`, `/audit-report`, `/audit-account`.

Step 3 - Configure the workshop portal

Edit `config/workshop.yml` (the first redemption flow can also auto-prompt for the URL if it is still the placeholder).

```
workshop_portal_url: "https://your-workshop-portal/..."
portal_kind:         "chatbot"     # chatbot | skillable | email
tenant_hint:         "your-label"
```

Step 4 - Redeem a workshop code (one-time per event)

```
/audit-account redeem
```

The redemption flow prompts for the code, navigates the portal, scrapes the issued credentials, signs in to `login.microsoftonline.com`, and DPAPI-encrypts the blob to `runtime/account/credential.enc`. Verify with `/audit-account show`.

Step 5 - Smoke-test the parser (no browser)

```
/audit-lab core-concepts-analytics-evaluations --dry-run
```

Step 6 - Single-lab run

```
/audit-lab core-concepts-analytics-evaluations
```

Expect 5-10 minutes. The plugin either appends a clean-pass entry to `runtime/audit-history.yml`, or files one GitHub issue + opens one fix-PR with the URLs printed in the summary.

Step 7 - Full event sweep

```
/audit-bootcamp                              # shortcut: event = bootcamp
/audit-event --event agent-buildathon-1month  # any other event by key
/audit-event                                 # interview picks the event
```

Expect 3-8 hours for the 11-lab bootcamp. The plugin checkpoints at every scene boundary. If it dies mid-run, resume with the printed run-id:

```
/audit-event --resume <run-id>
```

SECTION 5

KNOBS YOU MAY WANT TO TUNE

Configuration

config/workshop.yml

Workshop portal URL + redemption selectors. Edit on first install per your event organizer.

config/judge-config.yml — selected keys

Key	Default	Effect
confidence.min_to_include_in_issue	0.5	Findings below this never reach an issue.
confidence.low_confidence_marker_max	0.7	Findings in 0.5-0.7 are tagged 'low confidence' in the issue body.
execution.require_interactive_phase	true	Skip Phase 2 only when explicitly opted out.
execution.account_prompt_mode	always	Q1 interview behavior.
execution.network_retry_count	3	Connection-class failures retry up to N times before halting.
execution.fanout_concurrency	1	Parallel interactive browser sessions per account.
execution.static_fanout_concurrency	11	Background subagents for the static phase.
consistency.cross_lab_enabled	true	Enable the Phase 1.7 step 1a fan-in pass.
consistency.cross_lab_similarity_threshold	0.85	Threshold for near-match drift detection.
issues.pr_append.enabled_by_default	true	Screenshot-refresh commits onto open fix-PRs.
non_deterministic_lab_slugs	[agent-builder-m365, mcs-multi-agent]	Labs that default to log-only (no issue) unless --force-issue.
execution.model.preset	prompt	prompt | optimized | opus | custom. 'prompt' asks the Phase 1.5 Q2a model question.
execution.model.uc_subagent	sonnet	Per-UC subagent model (Playwright loop).
execution.model.judge	sonnet	Per-step LLM judge.
execution.model.critique	sonnet	Second-opinion critique pass.
execution.model.static_subagent	haiku	Per-lab static-phase fan-out subagent.
execution.model.cross_lab	sonnet	Cross-lab consistency fan-in.
execution.model.lab_parser	sonnet	Markdown -> step tree.
execution.model.issue_filer	haiku	Render issue body + gh issue create.

Key	Default	Effect
execution.model.fix_pr_filer	haiku	Apply suggested_correction diffs + open PR.
execution.model.pr_appender	haiku	Screenshot-only commit to open fix-PR branch.

Model preset (Phase 1.5 Q2a)

The orchestrator is always Opus (asserted at Phase 1 step 1). The `execution.model.preset` key controls sub-agent model selection:

- **prompt** (factory default) - Phase 1.5 Q2a asks the user which preset to use this run.
- **optimized** - each per-function key above is used as-is. Sonnet for UI reasoning + judge, Haiku for filers + static fan-out.
- **opus** - every per-function key is forced to opus.
- **custom** - read each per-function key literally (no preset rule). Use to mix-and-match.

Override per run with `--model-preset optimized|opus|custom` on any `/audit-*` command.

SECTION 6

WHEN THINGS GO SIDEWAYS

Troubleshooting

- **Slash commands don't appear** - restart Claude Code. Plugins are discovered at session start.
- **gh auth fails** - `gh auth status` must succeed and the user must have viewer permission on `microsoft/mcs-labs`.
- **credential.enc unreadable** - DPAPI keys are bound to (Windows user, machine). If you switched user or moved the machine, `/audit-account clear` then `/audit-account redeem` with a fresh code.
- **Workshop portal mismatch** - confirm `config/workshop.yml.portal_kind` matches the actual portal (chatbot vs Skillable vs email).
- **Lab file missing** - per-slug skip with `status: skipped, reason: lab_file_missing`. The whole run continues.
- **Stuck mid-lab** - every scene boundary writes `checkpoint.yml`. Resume with `/audit-event --resume <run-id>`.

SECTION 7

WHAT THE PLUGIN WILL NOT DO

Known limitations

- Windows-only (DPAPI).
- Single workshop-portal flow assumed (chatbot is the current default). Other portals require editing `config/workshop.yml.portal_kind` and the matching redemption reference.
- Screenshots aren't attached inline to issues - `gh issue create` doesn't support file uploads. Paths reference local run artifacts; maintainers can request the artifact for triage.
- No automatic tenant cleanup. Orphan agents created during audit runs accumulate; tenant hygiene is the user's responsibility.
- Cross-lab consistency in single-lab runs is discovery-limited - it can only diff against previously-audited siblings on disk.

SECTION 8

WHAT IT COSTS TO RUN

Projected token spend

These are field estimates calibrated against the bootcamp runs done during the May 2026 audit cycle. Token spend is dominated by the per-step LLM judge call - every other line item rounds to noise next to it.

Method

Measured one representative bootcamp lab end-to-end (`core-concepts-agent-knowledge-tools`): 77 executable steps after the parser-spec pass, across 4 use cases and 16 scenes. The 11-lab bootcamp totals ~720 judge-relevant steps. Other events are scaled by their step counts.

Per-step token budget

Each step's judge call assembles a structured prompt around the parsed instruction plus the live UI evidence. Typical breakdown:

Input segment	Tokens (avg)	Notes
Step instruction (raw_markdown + hints + sub-bullets)	~800	Bounded by parser output, ~109 chars per step measured.
CTX_VARS from prior UCs	~500	Agent names, schema names, knowledge URLs created upstream.
Before-snapshot (accessibility tree)	~2,000	Copilot Studio dialogs are dense; range 1.5-3k.
After-snapshot	~2,000	Same shape as before-snapshot.
Screenshot (vision)	~1,500	Token-equivalent for the page image.
Console + failed-network excerpts	~300	Usually small; large on broken steps.
Judge prompt scaffolding + JSON contract	~600	Stable across runs; benefits most from prompt cache.
TOTAL input per step	**~7,700**	Range observed: 6.5k - 9.5k.
TOTAL output per step	**~300**	Most steps pass (short verdict); findings add 200-400.

CRITIQUE PASS

Default-on. Only fires for non-pass verdicts (~10-20% of steps), so the realistic overhead is ~12% added input and ~10% added output across the run - not 100%. Disable in `config/judge-config.yml` with `critique.enabled: false` if you're cost-sensitive and willing to accept a higher false-positive rate.

Per-lab estimate

Lab	Steps (parsed)	Input tokens (M)	Output tokens (k)
agent-builder-m365	~50	0.50	25
core-concepts-agent-knowledge-tools (measured)	77	0.76	35
core-concepts-variables-agents-channels	~60	0.59	28
core-concepts-analytics-evaluations	~60	0.59	28
mcs-alm	~40	0.40	20
component-collections	~45	0.45	22
mcs-tools (largest)	~135	1.36	55
mcs-orchestration	~80	0.79	36
mcs-governance	~85	0.84	38
mcs-multi-agent	~75	0.74	34
autonomous-account-news	~40	0.40	20
Bootcamp total (11 labs)	~720	~7.4 M	~330 k

Includes: per-step judge calls + critique overhead + parser pass + static-phase fan-out + cross-lab consistency + UC handoff state files + per-lab issue and PR rendering + orchestrator coordination. Excludes: account redemption (one-time ~20k), interview (~5k), and Claude Code's own prompt-cache reuse (which materially lowers the realized cost - see "Prompt cache effect" below).

Per-event estimate

Event	Labs	Input tokens	Output tokens
bootcamp	11	~7.4 M	~330 k
agent-buildathon-1month	8	~5.4 M	~250 k
azure-ai-workshop	5	~3.4 M	~150 k
mcs-in-a-day	5	~3.0 M	~135 k
mcs-in-a-day-v2	4	~2.5 M	~115 k
agent-buildathon-1day	2	~1.0 M	~45 k

USD ranges (without prompt caching)

Multiplied through with model pricing as of late 2025. Prompt caching can cut realized input cost 50-90% on repeated reads of stable content (schema files, judge templates, parser spec) - see the next subsection.

Model	$ / Mtok in	$ / Mtok out	Per medium lab	Per bootcamp event
Opus 4.7	$15	$75	~$13-18	~$140
Sonnet 4.6	$3	$15	~$3-4	~$28
Haiku 4.5	$1	$5	~$1	~$9
Mixed (Opus orch + Sonnet judge)	-	-	~$5	~$50

PROMPT CACHE EFFECT

The Anthropic prompt cache has a 5-minute TTL. Within a single audit run, the parser spec, finding schema, judge template, and lab-config catalog are read repeatedly across every per-UC subagent and every per-step judge call. Empirically the cache hit rate for those segments lands in the 70-90% range, which trims realized input cost on Opus from ~$110 to roughly $30-60 for a full bootcamp run. The exact realized cost depends on how many subagents spawn in parallel (each one pays the cold cache once) and how much the browser session lingers on the same page between steps.

Knobs to reduce spend

- **--static-only** - skips Phase 2 entirely. Cuts ~95% of input tokens. Useful for doc-only sweeps where you just want spelling, link, image-ref, and cross-lab drift findings. Trade-off: misses live UI drift.
- **critique.enabled: false** in `config/judge-config.yml` - cuts ~10-15%. Raises false positives slightly; the issue body's "low confidence" markers compensate.
- **--model-preset optimized** (Phase 1.5 Q2a default) - the judge call dominates spend, so dropping the judge from Opus to Sonnet while keeping the orchestrator on Opus yields the best $-per-coverage trade. The `optimized` preset also drops the static fan-out + issue/PR filers to Haiku (they're text-only). One CLI flag swap; per-function overrides live in `config/judge-config.yml.execution.model.*`.

- **--labs csv** - cap scope to the labs you actually care about. Cost scales linearly with step count.
- **Reuse the cached account** - `account_prompt_mode: only_if_expired` avoids re-running redemption (~20k tokens). Accept the safety trade-off documented in Phase 1.5.
- **Resume rather than restart** - `/audit-event --resume <run-id>` skips finished UCs and replays their state files instead of re-judging.

BOTTOM LINE

Plan ~$30-$60 per full bootcamp audit run on Opus 4.7 with prompt caching, or ~$5-$12 per single-lab run. Sonnet-as-judge with Opus orchestrator drops those to ~$10-$15 and ~$2-$3 respectively. Static-only sweeps are effectively free (under $1 across all 11 labs). Token spend is dominated by per-step judge calls - the static and orchestrator phases are rounding error.

SECTION 9

OPERATIONAL REFERENCES

Read next

File	Read when...
skills/mcs-lab-auditor/SKILL.md	Full orchestrator procedure.
skills/mcs-lab-auditor/references/cross-lab-consistency.md	Cross-lab drift algorithm + finding format.
skills/mcs-lab-auditor/references/lab-parser-spec.md	Markdown -> step tree grammar + scene-fingerprint sidecar.
skills/mcs-lab-auditor/references/playwright-cookbook.md	Sign-in flow, scene-boundary auth probe, tool mapping per step kind.
skills/mcs-lab-auditor/references/finding-schema.md	Finding record schema + outcome/severity rubric.
docs/architecture.md	Architecture deep-dive with Mermaid diagrams.
docs/installation.md	Full step-by-step install with troubleshooting.
docs/extending.md	Adapting to a new event, portal, or repo location.
docs/security.md	What is encrypted, what is logged, what is at risk.
CHANGELOG.md	Version history.